Đại học Quốc gia TP. Hồ Chí Minh
Trường Đại học Bách Khoa
Khoa Khoa học và Kỹ thuật Máy tính

Môn học: Hệ Điều Hành (Operating Systems)

Bài tập lớn – Hệ Điều Hành Đơn Giản

(Assignment – Simple Operating System / Phiên bản demacOprx)

Ngày 17 tháng 10 năm 2026

(Bản dịch Tiếng Việt đầy đủ kèm chú giải phục vụ học tập & làm việc nhóm)

Mục tiêu (Goals): Mục tiêu của bài tập lớn này là mô phỏng các thành phần trọng yếu của một hệ điều hành đơn giản, chẳng hạn như bộ lập lịch (*scheduler*), các cơ chế đồng bộ hóa (*synchronization mechanisms*), cũng như các thao tác trên bộ nhớ vật lý và bộ nhớ ảo (*physical and virtual memory operations*).

Nội dung (Contents): Cụ thể, sinh viên sẽ thực hành với ba phân hệ (module) chính: bộ lập lịch, đồng bộ hóa, và cơ chế cấp phát bộ nhớ từ bộ nhớ ảo sang bộ nhớ vật lý:

- Bộ lập lịch (*scheduler*)
- Đồng bộ hóa (*synchronization*)
- Các thao tác cấp phát bộ nhớ từ bộ nhớ ảo sang bộ nhớ vật lý (*memory-allocation operations from virtual to physical memory*)

Ngoài ra, sinh viên sẽ thực hành thiết kế và hiện thực một giao diện lập trình hệ điều hành đơn giản thông qua các lời gọi hệ thống (*system calls*).

Kết quả đạt được (Results): Sau bài tập lớn này, sinh viên sẽ nắm bắt được các nguyên lý cốt lõi của một hệ điều hành đơn giản, đồng thời có khả năng giải thích và minh họa vai trò của từng phân hệ chính trong hệ điều hành.

Mục lục

1 Giới thiệu (Introduction) 4

1.1 Tổng quan (An overview) . . . 4
1.2 Tổ chức Mã nguồn (Source Code) . . . 5
1.3 Tiến trình (Processes) . . . 6
1.4 Làm thế nào để tạo một Tiến trình? (How to Create a Process?) . . . 9
1.5 Làm thế nào để chạy Mô phỏng? (How to Run the Simulation) . . . 9
1.6 Làm thế nào để viết Giao diện Nhân? (How to Write the Kernel Interface) . . . 10
1.6.1 Cấu trúc nhân (`struct krnl_t`) . . . 10
1.6.2 Lời gọi hệ thống (The system call) . . . 10
1.6.3 Hướng dẫn thêm một lời gọi hệ thống mới vào Simple OS . . . 12

2 Hiện thực (Implementation) 14

2.1 Bộ lập lịch (Scheduler) . . . 14
2.2 Quản lý Bộ nhớ (Memory Management) . . . 15
2.2.1 Bố cục bộ nhớ của tiến trình (The process memory layout) . . . 15
2.2.2 Ánh xạ bộ nhớ ảo trong mỗi tiến trình (Virtual memory mapping in each process) . . . 17
2.2.3 Bộ nhớ vật lý của hệ thống (The system's physical memory) . . . 19
2.2.4 Cơ chế dịch địa chỉ dựa trên phân trang (Paging-based address translation scheme) . . . 20
2.2.5 Tổng kết các phần hiện thực hướng phân trang (Wrapping-up all paging-oriented implementations) . . . 21
2.3 Bộ đồng xử lý (Co-processors) . . . 22
2.3.1 Hoạt động bất đồng bộ và địa chỉ bus (Asynchronous operation and bus address) 22
2.3.2 Truy cập bộ nhớ trực tiếp – DMA (Direct Memory Access) . . . 22
2.3.3 Bộ xử lý đồ họa – GPU & Các công nghệ tăng tốc I/O . . . 24
2.4 Kết hợp tất cả lại & Đồng bộ hóa (Put It All Together) . . . 25

3 Nộp bài và Đánh giá (Submission) 27

3.1 Mã nguồn (Source code) . . . 27
3.2 Tóm tắt các Yêu cầu Hiện thực (Requirements) . . . 27
3.3 Báo cáo (Report) . . . 27
3.4 Thang điểm Đánh giá (Grading) . . . 28
3.5 Quy tắc đạo đức & Bản quyền (Code of ethics) . . . 28

A Phụ lục: Bố cục Địa chỉ Linux (Linux Address Layout) 30

B Phụ lục: Các vùng nhớ ảo macOS (MacOS Virtual Memory Regions) **31**

C Phụ lục: Bố cục Địa chỉ NTKernel (Windows NTKernel Address Layout) **31**

D Phụ lục: Nguyên mẫu phần mềm của bộ máy DMA (Software prototype of the DMA engine) **32**

1 Giới thiệu (Introduction)

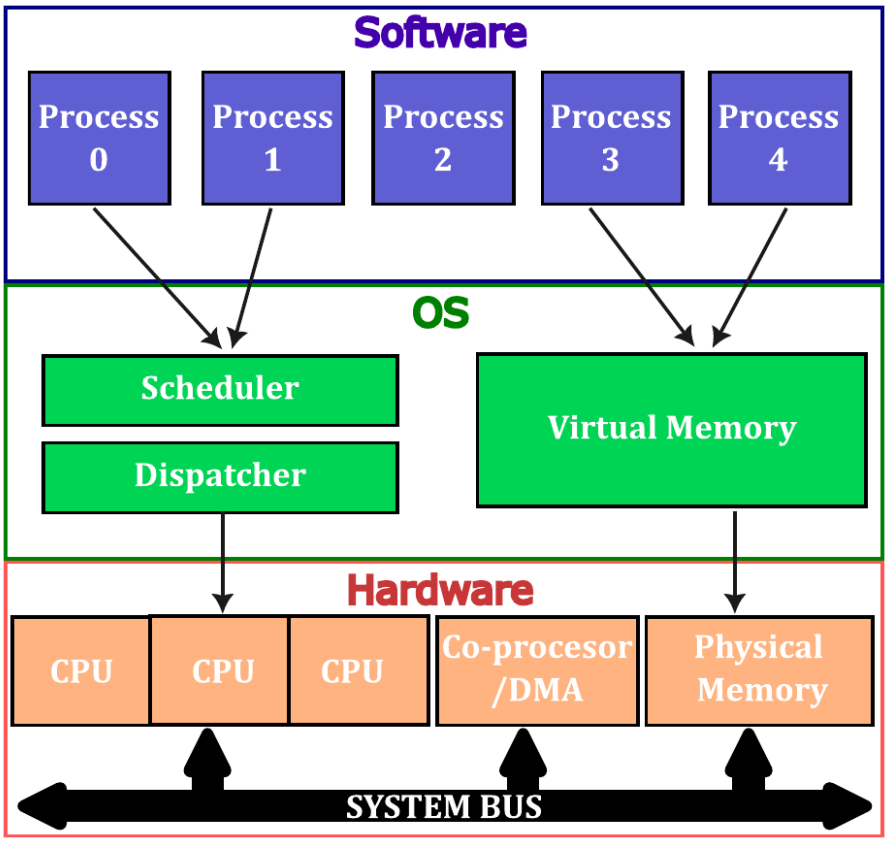
1.1 Tổng quan (An overview)

Bài tập lớn này mô phỏng một hệ điều hành đơn giản nhằm giúp sinh viên hiểu rõ các khái niệm nền tảng về lập lịch (*scheduling*), đồng bộ hóa (*synchronization*), và quản lý bộ nhớ (*memory management*). Hình 1 thể hiện kiến trúc tổng thể của hệ điều hành mà chúng ta sẽ hiện thực. Nhìn chung, hệ điều hành quản lý hai tài nguyên ảo hóa là **CPU / Bộ đồng xử lý (co-processor)** và **RAM**, thông qua các thành phần cốt lõi sau:

i. Bộ lập lịch và Bộ điều phối (Scheduler and dispatcher): Xác định tiến trình (*process*) nào được phép chạy trên CPU nào tại mỗi thời điểm.

ii. Bộ nhớ ảo (Virtual memory): Cô lập không gian bộ nhớ của từng tiến trình với các tiến trình khác. Bộ nhớ RAM vật lý được chia sẻ bởi nhiều tiến trình, nhưng mỗi tiến trình hoàn toàn không biết đến sự tồn tại của các tiến trình còn lại. Điều này đạt được bằng cách cấp cho mỗi tiến trình một không gian địa chỉ ảo riêng biệt; bộ máy quản lý bộ nhớ ảo (*Virtual Memory Engine*) sẽ chịu trách nhiệm ánh xạ và dịch các địa chỉ ảo do tiến trình cung cấp sang các địa chỉ vật lý tương ứng trên RAM.

iii. Bộ đồng xử lý (Co-processor): Là một bộ vi xử lý phần cứng phụ trợ được thiết kế để bổ sung chức năng cho bộ vi xử lý chính (CPU). Bằng cách chuyển giao (*offload*) các tác vụ chuyên biệt sang phần cứng chuyên dụng, bộ đồng xử lý giúp giải phóng CPU để tập trung vào việc quản lý hệ thống chung, từ đó tăng tốc đáng kể hiệu năng toàn hệ thống.



Hình 1: Góc nhìn tổng quan về các phân hệ chính trong bài tập lớn (The general view of key modules in this assignment)

Ví dụ: Bộ máy truy cập bộ nhớ trực tiếp (**DMA – Direct Memory Access**) truyền dữ liệu giữa

bộ nhớ và thiết bị ngoại vi với sự can thiệp tối thiểu từ CPU; bộ xử lý đồ họa (**GPU**) tăng tốc xử lý đồ họa và các phép toán vector/ma trận; bộ điều hợp mạng (*network adapter*) có thể tăng tốc các thao tác TCP như tính checksum và phân mảnh gói tin. Việc chuyển giao các tác vụ này giúp CPU rảnh tay để tập trung vào các công việc hướng người dùng.

Thiết kế này hỗ trợ **hoạt động chế độ kép (dual-mode operation)**, đảm bảo tính bảo vệ hệ thống và kiểm soát chặt chẽ quyền truy cập tài nguyên phần cứng. Hệ điều hành hoạt động ở hai chế độ: **chế độ người dùng (user mode)** và **chế độ nhân (kernel mode)**, được phân biệt bởi một bit chế độ (*mode bit*) do phần cứng hỗ trợ nhằm cho biết CPU đang thực thi mã người dùng hay mã đặc quyền của nhân. Cơ chế này đảm bảo các chương trình người dùng không thể tự ý thực thi các lệnh đặc quyền, giữ vững tính ổn định và bảo mật của hệ thống.

Thông qua các phân hệ này, hệ điều hành cho phép nhiều tiến trình do người dùng tạo ra cùng chia sẻ và sử dụng tài nguyên tính toán ảo hóa. Do đó, trong bài tập lớn này, chúng ta tập trung hiện thực **bộ lập lịch/điều phối**, **bộ máy quản lý bộ nhớ ảo**, và **giao tiếp với bộ đồng xử lý DMA**.

1.2 Tổ chức Mã nguồn (Source Code)

Sau khi tải mã nguồn của bài tập lớn từ mục *Resource* trên trang LMS và giải nén, bạn sẽ thấy mã nguồn được tổ chức như sau:

- **Các tập tin tiêu đề (Header files – thư mục `include/`):**
  - `timer.h`: Định nghĩa bộ định thời (*timer*) cho toàn hệ thống.
  - `cpu.h`: Định nghĩa các hàm dùng để hiện thực CPU ảo.
  - `queue.h`: Định nghĩa cấu trúc và hàm hiện thực hàng đợi (*queue*) chứa các khối PCB của tiến trình.
  - `sched.h`: Định nghĩa các hàm được sử dụng bởi bộ lập lịch (*scheduler*).
  - `mem.h`: *(Đã lỗi thời – obsolete)* Định nghĩa các hàm của bộ máy bộ nhớ ảo phiên bản cũ.
  - `loader.h`: Định nghĩa các hàm của bộ nạp (*loader*) để nạp chương trình từ đĩa vào bộ nhớ.
  - `common.h`: Định nghĩa các cấu trúc (`struct pcb_t`, `struct krnl_t`...) và hàm dùng chung xuyên suốt HĐH.
  - `bitops.h`: Định nghĩa các macro thao tác trên dữ liệu bit (`BIT`, `GENMASK`, `GETVAL`, `SETVAL`...).
  - `os-mm.h`, `mm.h`, `mm64.h`: Định nghĩa các cấu trúc và dữ liệu cơ sở cho quản lý bộ nhớ theo cơ chế phân trang (*paging-based memory management*).
  - `syscall.h`: Định nghĩa tiêu đề cho lời gọi hệ thống (*system call*).
  - `libmem.h`: Định nghĩa tiêu đề thư viện chuẩn thao tác bộ nhớ.
  - `dma.h`, `dpi_rtl.h`, `sysbus.h`, `Vdma_top*.h`: Định nghĩa giao diện điều khiển bộ đồng xử lý DMA (Verilator RTL) và bus hệ thống.
  - `os-cfg.h`: Định nghĩa các hằng số cấu hình dùng để bật/tắt các tính năng phần mềm.
- **Các tập tin mã nguồn (Source files – thư mục `src/`):**
  - `timer.c`: Hiện thực bộ định thời hệ thống.
  - `cpu.c`: Hiện thực CPU ảo thực thi từng lệnh của tiến trình.
  - `queue.c`: Hiện thực các thao tác trên hàng đợi (ưu tiên).

  - `paging.c`, `mem.c`: *(Đã lỗi thời – obsolete)* Thiết kế RAM và bộ nhớ ảo phiên bản cũ.
  - `dma_c_testbench.cpp`, `dpi_rtl.cpp`, `sysbus.c`, `Vdma_top*.cpp`: Hiện thực bộ máy DMA phần cứng (Verilator RTL) và bus hệ thống.
  - `os.c`: Chứa hàm `main()` khởi động toàn bộ hệ điều hành, tạo các luồng CPU, Loader và Co-processor.
  - `loader.c`: Hiện thực bộ nạp chương trình từ file mô tả vào PCB.
  - `sched.c`: Hiện thực bộ lập lịch tiến trình (MLQ Scheduler).
  - `mm.c`, `mm64.c`, `mm-vm.c`, `mm-memphy.c`: Hiện thực quản lý bộ nhớ dựa trên phân trang (32-bit/64-bit, vùng nhớ ảo và thiết bị vật lý RAM/SWAP).
  - `libmem.c`, `libstd.c`: Hiện thực thư viện chuẩn phía không gian người dùng (*userspace*) và các hàm hỗ trợ bộ nhớ.
  - `syscall.c`, `syscall.tbl`, `syscalltbl.sh`, `sys_mem.c`, `sys_listsyscall.c`: Hiện thực cơ chế lời gọi hệ thống.

- `Makefile`: Tập tin kịch bản biên dịch dự án.
- `input/`: Thư mục chứa tập hợp các file cấu hình phần cứng và chương trình mẫu (`input/proc/`) dùng để kiểm thử.
- `output/`: Thư mục chứa kết quả đầu ra mẫu của hệ thống để đối chiếu.

1.3 Tiến trình (Processes)

Chúng ta sẽ xây dựng một hệ điều hành đa nhiệm (*multitasking OS*) cho phép nhiều tiến trình chạy đồng thời, vì vậy cần hiểu rõ cách tổ chức của một tiến trình. Hệ điều hành quản lý các tiến trình thông qua **Khối điều khiển tiến trình (PCB – Process Control Block)**, được mô tả như sau:

```
// Trich tu include/common.h
struct pcb_t {
    uint32_t pid;
    uint32_t priority;
    char path[100];
    struct code_seg_t * code;
    addr_t regs[10];
    uint32_t pc;
#ifdef MLQ_SCHED
    uint32_t prio;
#endif
    struct mm_struct *mm;
    struct krnl_t *krnl;
    struct page_table_t * page_table;
    uint32_t bp;
};
```

Ý nghĩa của các trường trong `struct pcb_t`:

- `pid`: Định danh của tiến trình (*Process ID*).
- `priority`: Độ ưu tiên mặc định của tiến trình; **giá trị càng nhỏ thì độ ưu tiên càng cao**. Giá trị lịch sử này phụ thuộc vào thuộc tính của file chương trình và giữ nguyên không đổi trong suốt quá trình thực thi.
- `code`: Phân đoạn mã lệnh (*Text segment*) của tiến trình. (Để đơn giản hóa mô phỏng, chúng ta không đặt phân đoạn mã lệnh này vào trong RAM).

- `regs`: Các thanh ghi (*Registers*); mỗi tiến trình có thể sử dụng tối đa 10 thanh ghi được đánh số từ `0` đến `9` (thường dùng làm chỉ số vùng nhớ/biến `rgid`).
- `pc`: Giá trị hiện tại của bộ đếm chương trình (*Program Counter*), trỏ tới lệnh tiếp theo sẽ thực thi.
- `page_table`: Bảng dịch từ địa chỉ ảo sang địa chỉ vật lý *(đã lỗi thời từ phiên bản cũ, không sử dụng)*.
- `bp`: Con trỏ ngắt (*break pointer*), dùng để quản lý phân đoạn heap.
- `prio`: Độ ưu tiên thực tế khi thực thi (*live priority*, nếu bật `MLQ_SCHED`); giá trị này sẽ ghi đè lên độ ưu tiên mặc định (`priority`).
- `mm` và `krnl`: Con trỏ tới cấu trúc quản lý bộ nhớ phân trang (`mm_struct`) của tiến trình và cấu trúc nhân (`krnl_t`).

Tương tự như một tiến trình thực tế, mỗi tiến trình trong mô phỏng này là một danh sách các câu lệnh (*instructions*) được CPU thực thi tuần tự từng lệnh một từ đầu đến cuối (chúng ta không hiện thực lệnh nhảy *jump* ở đây). Có **9 câu lệnh** mà một tiến trình có thể thực hiện:

- `CALC`: Thực hiện một phép tính toán nào đó sử dụng CPU. Lệnh này không nhận tham số.

  > ***Chú giải về Vùng nhớ (Memory region):*** *Một vùng nhớ là một khu vực lưu trữ được cấp phát cho một biến. Thuật ngữ này gắn liền với một chỉ số (index) trong* ***BẢNG KÝ HIỆU (SYMBOL TABLE)*** *và thường được hiển thị dễ đọc cho con người thông qua tên biến và cơ chế ánh xạ của trình biên dịch. Tuy nhiên, cơ chế ánh xạ tên biến thuộc phạm vi môn Trình biên dịch (Compiler) chứ không nằm trong phạm vi môn Hệ điều hành. Để đơn giản, trong bài tập này chúng ta tham chiếu trực tiếp đến một vùng nhớ (biến) thông qua chỉ số thanh ghi (`reg` / `rgid`), và mỗi chương trình/tiến trình có giới hạn số lượng biến tối đa là `PAGING_MAX_SYMTBL_SZ = 30`.*

- `ALLOC`: Cấp phát một khối byte trên bộ nhớ chính (RAM – không gian người dùng). Cú pháp lệnh:

  ```
  alloc [size] [reg]
  ```

  Trong đó `size` là số byte mà tiến trình muốn cấp phát từ RAM, và `reg` là số hiệu thanh ghi (kiêm chỉ số vùng nhớ trong `symrgtbl`) sẽ lưu địa chỉ của byte đầu tiên của vùng nhớ được cấp phát. Ví dụ: lệnh `alloc 124 7` xin cấp phát 124 byte từ HĐH, và địa chỉ byte đầu tiên của vùng 124 byte đó sẽ được lưu vào thanh ghi số `#7`.

- `KMALLOC`: Cấp phát một khối byte trên **không gian bộ nhớ nhân (kernel memory space)**. Cú pháp lệnh:

  ```
  kmalloc [size] [reg]
  ```

  Trong đó `size` là số byte muốn cấp phát và `reg` là số hiệu thanh ghi lưu địa chỉ byte đầu tiên của vùng nhớ được cấp phát. Lệnh này cấp phát bộ nhớ liên tục về mặt vật lý (*physically contiguous memory*). HĐH phải tìm một khối bộ nhớ vật lý liên tục.

- `FREE`: Giải phóng vùng nhớ đã được cấp phát trước đó. Cú pháp lệnh:

```
free [reg]
```

Trong đó `reg` là số hiệu thanh ghi đang giữ địa chỉ byte đầu tiên của vùng nhớ cần thu hồi.

- `READ` *(chỉ dành cho không gian người dùng – userspace only)*: Đọc 1 byte từ bộ nhớ và chỉ được phép truy cập địa chỉ thuộc không gian người dùng (*userspace*). Lệnh này ngăn chặn việc truy cập trái phép vào địa chỉ không gian nhân (*kernelspace*). Cú pháp lệnh:

```
read [source] [offset] [destination]
```

Lệnh này đọc 1 byte bộ nhớ tại địa chỉ bằng `(giá trị của thanh ghi source + offset)` và lưu giá trị đọc được vào thanh ghi `destination`. Ví dụ: giả sử giá trị của thanh ghi `#1` là `0x123`, khi đó lệnh `read 1 20 2` sẽ đọc 1 byte tại địa chỉ `0x123 + 0x14` (`0x14` là 20 ở hệ thập lục phân) và lưu vào thanh ghi `#2`.

- `WRITE` *(chỉ dành cho không gian người dùng – userspace only)*: Ghi một giá trị 1 byte vào bộ nhớ và chỉ được phép truy cập địa chỉ thuộc không gian người dùng. Cú pháp lệnh:

```
write [data] [destination] [offset]
```

Lệnh này ghi giá trị `data` vào địa chỉ bằng `(giá trị của thanh ghi destination + offset)`. Ví dụ: giả sử giá trị thanh ghi `#1` là `0x123`, lệnh `write 10 1 20` sẽ ghi giá trị `10` vào ô nhớ tại địa chỉ `0x123 + 0x14`.

- `COPY_FROM_USER`: Thực hiện đọc trực tiếp dữ liệu từ địa chỉ không gian người dùng (*userspace*) và ghi sang địa chỉ không gian nhân (*kernelspace*). Cú pháp lệnh:

```
copy_from_user [source] [destination] [offset] [size]
```

Lệnh thực hiện truyền dữ liệu từ user space sang kernel space: đọc dữ liệu từ vùng nhớ `source` (user) và ghi vào vùng nhớ `destination` (kernel) bắt đầu từ độ lệch `offset` với kích thước `size` byte.

- `COPY_TO_USER`: Thực hiện đọc trực tiếp dữ liệu từ địa chỉ không gian nhân (*kernelspace*) và ghi sang địa chỉ không gian người dùng (*userspace*). Cú pháp lệnh:

```
copy_to_user [source] [destination] [offset] [size]
```

Lệnh thực hiện truyền dữ liệu từ kernel space sang user space: đọc dữ liệu từ vùng nhớ `source` (kernel) và ghi vào vùng nhớ `destination` (user) bắt đầu từ độ lệch `offset` với kích thước `size` byte.

- `DMA_MEMCPY`: Thực hiện sao chép trực tiếp nội dung bộ nhớ từ vùng nhớ nguồn (`source`) sang vùng nhớ đích (`destination`) tại cùng độ lệch `offset` thông qua bộ điều khiển phần cứng DMA. Cú pháp lệnh:

```
dma_memcpy [source] [destination] [offset] [size]
```

Lệnh truyền dữ liệu từ vùng `source` sang vùng `destination` bắt đầu từ `offset` với độ dài `size` byte.

1.4 Làm thế nào để tạo một Tiến trình? (How to Create a Process?)

Nội dung của mỗi tiến trình thực chất là một bản sao của một chương trình lưu trên đĩa cứng. Do đó, để tạo một tiến trình, trước tiên chúng ta phải tạo file chương trình mô tả nội dung của nó. Một chương trình được định nghĩa bởi một file văn bản duy nhất theo định dạng sau:

```
[priority] [N = so luong cau lenh (instructions)]
instruction 0
instruction 1
...
instruction N-1
```

Trong đó `priority` là độ ưu tiên mặc định của tiến trình được tạo ra từ chương trình này. Cần lưu ý rằng hệ thống này sử dụng **cơ chế độ ưu tiên kép (dual priority mechanism)**:

- Độ ưu tiên càng cao (tức **giá trị số càng nhỏ**), tiến trình càng có nhiều cơ hội được CPU chọn ra từ hàng đợi để thực thi (xem Mục 2.1 để biết chi tiết).
- `N` là tổng số câu lệnh, và `N` dòng tiếp theo là các câu lệnh được biểu diễn theo cú pháp đã trình bày ở Mục 1.3. Bạn có thể mở các file trong thư mục `input/proc/` (như `m0s`, `m2s`, `p0s`...) để xem các chương trình mẫu.
- **Cơ chế độ ưu tiên kép:** Giá trị `priority` mặc định trong file chương trình sẽ bị **ghi đè (overwrite)** bởi độ ưu tiên trực tiếp (*live priority* – trường `prio`) được truyền vào lúc nạp tiến trình trong file cấu hình môi trường. Điều này rất phổ biến trong các HĐH hiện đại: mỗi chương trình có một mức ưu tiên mặc định (từ tác vụ người dùng ít quan trọng đến tiến trình ngầm hệ thống quan trọng), nhưng người dùng có thể chạy chương trình với một độ ưu tiên khác (tương tự như bấm chuột phải vào tiến trình trong Task Manager để đổi Priority hoặc chạy với quyền Administrator).

1.5 Làm thế nào để chạy Mô phỏng? (How to Run the Simulation)

Nhiệm vụ của chúng ta trong bài tập lớn này là hiện thực một HĐH đơn giản và mô phỏng nó trên phần cứng ảo. Để bắt đầu quá trình mô phỏng, ta cần một file mô tả trong thư mục `input/` quy định thông số phần cứng và kịch bản các tiến trình sẽ chạy. Định dạng cơ bản của file mô tả như sau:

```
[time slice] [N = So luong CPU] [M = So luong tien trinh se chay]
[time 0] [path 0] [priority 0]
[time 1] [path 1] [priority 1]
...
[time M-1] [path M-1] [priority M-1]
```

(Lưu ý: Khi tắt cờ `MM_FIXED_MEMSZ` trong `os-cfg.h`, dòng thứ 2 sẽ có thêm dòng khai báo dung lượng RAM và 4 thiết bị SWAP – xem chi tiết tại Mục 2.2.5).

Trong đó:

- `time slice`: Khoảng thời gian định mức (tính theo đơn vị time slot) mà một tiến trình được phép chạy liên tục trên CPU trước khi bị thu hồi CPU.
- `N`: Số lượng CPU có sẵn trong hệ thống.
- `M`: Tổng số tiến trình sẽ được nạp và chạy.
- `time i`: Thời điểm (time slot) mà tiến trình thứ i được bộ Loader nạp vào hệ thống.

- `path i`: Tên file mô tả chương trình nằm trong thư mục `input/proc/`.
- `priority i`: Độ ưu tiên thực tế (*live priority*) khi tiến trình được gọi chạy, ghi đè lên độ ưu tiên mặc định trong file chương trình.

Để khởi chạy mô phỏng, trước tiên bạn phải biên dịch mã nguồn bằng lệnh `make all`, sau đó chạy lệnh:

```
./os [configure_file]
```

Trong đó `configure_file` là tên của file cấu hình đặt trong thư mục `input/` (ví dụ: `./os os_1_singleCPU_mlq_p`

1.6 Làm thế nào để viết Giao diện Nhân? (How to Write the Kernel Interface)

1.6.1 Cấu trúc nhân (`struct krnl_t`)

Cấu trúc `struct krnl_t` đóng gói các thành phần cốt lõi của nhân hệ điều hành nhằm quản lý việc lập lịch tiến trình và phân hệ bộ nhớ. Nó tích hợp các đơn vị quản lý bộ nhớ (MMU) và các trừu tượng hóa bộ nhớ vật lý, cho phép xử lý hiệu quả RAM và nhiều không gian hoán đổi (SWAP):

```
// Trich tu include/common.h
/* Kernel structure */
struct krnl_t
{
    struct queue_t *ready_queue;
    struct queue_t *running_list;
#ifdef MLQ_SCHED
    struct queue_t *mlq_ready_queue;
#endif
#ifdef MM_PAGING
    struct mm_struct *mm;
    struct memphy_struct *mram;
    struct memphy_struct **mswp;
    struct memphy_struct *active_mswp;
    uint32_t active_mswp_id;
    struct kcache_pool_struct *kcpooltbl;
#ifdef MM64
    addr_t *krnl_pgd;
    addr_t *krnl_p4d;
    addr_t *krnl_pud;
    addr_t *krnl_pmd;
    addr_t *krnl_pt;
#else
    uint32_t *krnl_pgd;
#endif
#endif
};
```

1.6.2 Lời gọi hệ thống (The system call)

Lời gọi hệ thống (*system call*) là giao diện nền tảng giữa một ứng dụng người dùng và nhân của Hệ điều hành Đơn giản.

Lời gọi hệ thống và các hàm bọc thư viện (Library wrapper functions): Thông thường, các chương trình ứng dụng không gọi trực tiếp lời gọi hệ thống mà gọi thông qua các hàm bọc

(*wrapper functions*) trong thư viện `libstd` (hoặc `libmem`). Các hàm bọc trong `libstd` thường rất mỏng, chỉ làm nhiệm vụ chép các đối số vào các thanh ghi tương ứng (`struct sc_regs`) trước khi kích hoạt lời gọi hệ thống chuyển vào nhân. Chi tiết quá trình gọi lời gọi hệ thống được minh họa ở Hình 2.

Danh sách lời gọi hệ thống có sẵn trong Simple OS:

```
System call          Kernel      Notes
-------------------------------------------------
listsyscall          3.0
memmap               3.0
```

Trang hướng dẫn sử dụng (Manual page) của các System call:

1. `listsyscall` – Liệt kê tất cả các lời gọi hệ thống (Số hiệu: `SYSCALL 0`).
2. `memmap` – Thao tác ánh xạ bộ nhớ (Số hiệu: `SYSCALL 17`):

```
Name
    memmap - map memory
Synopsis
    SYSCALL 17 SYSMEM_OP REG_ARG2 REG_ARG3
Description
    memmap ho tro cac thao tac anh xa bo nho sau:
    + SYSMEM_MAP_OP   voi ham xu ly vmap_pgd_memset()
    + SYSMEM_INC_OP   voi ham xu ly inc_vma_limit()
    + SYSMEM_SWP_OP   voi ham xu ly __mm_swap_page()
    + SYSMEM_IO_READ  voi ham xu ly MEMPHY_read()
    + SYSMEM_IO_WRITE voi ham xu ly MEMPHY_write()
```

Câu hỏi 1 (Trang 9 đề gốc)

Ưu điểm và nhược điểm của việc sử dụng một giao diện lời gọi hệ thống thống nhất (*unified system call interface*) để thao tác trên nhiều thành phần hệ thống khác nhau (ví dụ: đọc/ghi/giải phóng cho tập tin, bộ nhớ và thiết bị I/O) là gì? Trong phân tích của bạn, hãy xem xét cách sự trừu tượng hóa này ảnh hưởng đến thiết kế hệ điều hành, sự đánh đổi về hiệu năng, độ phức tạp khi xử lý lỗi, và sự cân bằng giữa tính khả chuyển (*portability*) với tính hiệu quả (*efficiency*).

Gợi ý trả lời Câu hỏi 1 cho Báo cáo

- **Ưu điểm:** Đơn giản hóa API cho lập trình viên (triết lý *"Everything is a file"* trong UNIX), tăng tính bảo mật vì mọi truy cập phần cứng đều đi qua một cổng kiểm soát duy nhất (chuyển từ User mode sang Kernel mode), và giúp mã nguồn ứng dụng dễ dàng mang sang các phần cứng khác (*portability*).
- **Nhược điểm & Đánh đổi:** Tốn chi phí chuyển ngữ cảnh (*context switch overhead*) và kiểm tra tham số chung; một giao diện quá tổng quát (như gom cả 5 thao tác `SYSMEM_*` vào chung `SYSCALL 17`) khiến việc truyền tham qua thanh ghi (`a1, a2, a3`) kém tường minh, khó tối ưu hóa chuyên sâu cho từng thiết bị tốc độ cao, và việc phân loại mã lỗi (*error handling*) phức tạp hơn.

1.6.3 Hướng dẫn thêm một lời gọi hệ thống mới vào Simple OS

Bước 1 – Khởi tạo (Creation):

1. Tạo một tập tin C cho lời gọi hệ thống mới tại `src/sys_xxxhandler.c`:

```c
// Tu src/sys_xxxhandler.c
#include "common.h"
#include "syscall.h"
#include "stdio.h"

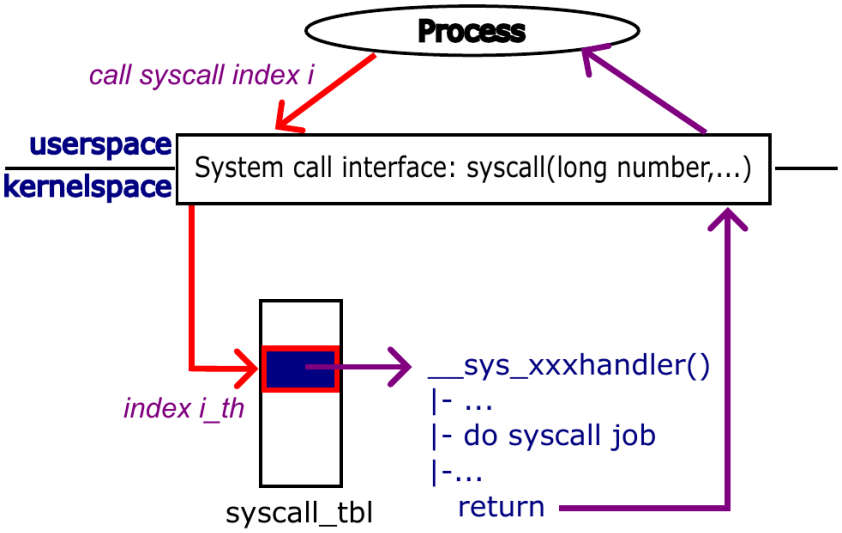
int __sys_xxxhandler(struct krnl_t *krnl, uint32_t pid, struct sc_regs
    * regs)
{
    /* TODO: implement syscall job */
    printf("The first system call parameter %lu\n", regs->a1);
    return 0;
}
```

2. Thêm đường dẫn đối tượng vào `Makefile` (với số hiệu duy nhất, ví dụ `440`):

```
# Tu Makefile
SYSCALL_OBJ += $(addprefix $(OBJ)/, sys_xxxhandler.o)
```

3. Đăng ký lời gọi hệ thống vào bảng `src/syscall.tbl`:

```
# Tu src/syscall.tbl
440     xxx     sys_xxxhandler
```



Hình 2: Quá trình xử lý khi ứng dụng người dùng gọi một lời gọi hệ thống (The handling of user application invoking a system call)

Bước 2 – Cài đặt (Installation): Biên dịch bằng `$ make all` và chạy `$ ./os <input_file>`.

Bước cuối – Kiểm tra kết quả (Checking results): Tạo chương trình `sc` trong `input/proc/` gọi `syscall 440 1` và file cấu hình `os_syscall`, khi chạy `./os os_syscall` sẽ in ra dòng `The first system call parameter 1`.

Câu hỏi 2 (Trang 11 đề gốc)

Khi một lời gọi hệ thống (*system call*) thực thi quá lâu, Hệ điều hành phát hiện và xử lý trường hợp này như thế nào?

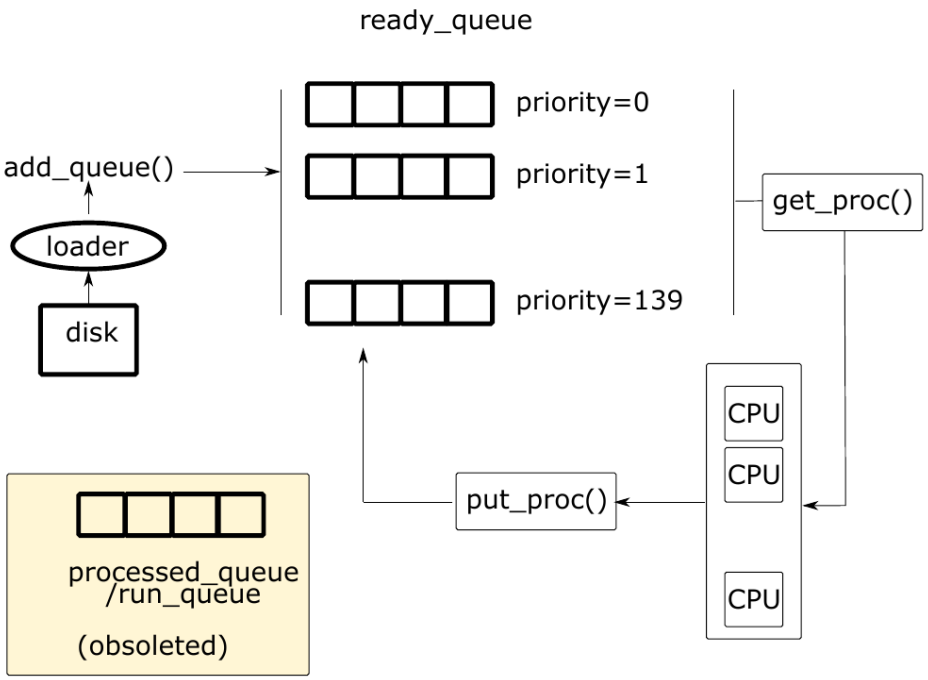
Gợi ý trả lời Câu hỏi 2 cho Báo cáo

Hệ điều hành phát hiện thông qua **ngắt đồng hồ phần cứng (hardware timer interrupt / watchdog timer)**. Mỗi khi đồng hồ điểm nhịp (*tick*), trình xử lý ngắt sẽ kiểm tra thời gian tiến trình đã tiêu tốn. Nếu nhân hỗ trợ **tiếm quyền trong nhân (preemptive kernel)**, bộ lập lịch sẽ tạm dừng tiến trình đó khi hết `time_slice` hoặc khi chờ I/O để nhường CPU cho tiến trình khác; nếu bị treo vô hạn (deadlock/timeout), watchdog hoặc tín hiệu hệ thống sẽ hủy tác vụ và trả về mã lỗi timeout (như `-ETIMEDOUT` hoặc `-EINTR`).

2 Hiện thực (Implementation)

2.1 Bộ lập lịch (Scheduler)

Trước tiên, chúng ta hiện thực bộ lập lịch. Hình 3 mô tả cách hệ điều hành lập lịch cho các tiến trình. Hệ điều hành được thiết kế để hoạt động trên nhiều bộ vi xử lý (đa CPU). HĐH sử dụng nhiều hàng đợi được gọi là `ready_queue` để xác định tiến trình nào sẽ được thực thi khi có một CPU rảnh. Mỗi hàng đợi gắn liền với một giá trị độ ưu tiên cố định. Bộ lập lịch được thiết kế dựa trên thuật toán **"Hàng đợi đa mức" (Multi-Level Queue – MLQ)** được sử dụng trong nhân Linux[1].



Hình 3: Hoạt động của bộ lập lịch trong bài tập lớn (The operation of scheduler in the assignment)

Theo Hình 3, bộ lập lịch hoạt động như sau:

- Đối với mỗi chương trình mới, bộ nạp (`loader`) sẽ tạo một tiến trình mới và gán một khối `pcb_t` mới cho nó.
- Bộ nạp đọc và sao chép nội dung chương trình vào phân đoạn mã lệnh (`code` pointer trong PCB).
- Khối PCB của tiến trình được đẩy vào hàng đợi `mlq_ready_queue` tương ứng có cùng độ ưu tiên với giá trị `prio` của tiến trình đó, rồi chờ CPU.
- CPU chạy các tiến trình theo phong cách xoay vòng (**Round-Robin**). Mỗi tiến trình được phép chạy tối đa trong một khoảng `time_slice`. Hết `time_slice` mà chưa xong, CPU buộc phải đưa tiến trình đó trở lại hàng đợi `mlq_ready_queue` ứng với độ ưu tiên của nó (`put_proc`), rồi lấy một tiến trình khác từ `ready_queue` ra chạy tiếp (`get_proc`).

Mô tả chính sách MLQ (Multi-Level Queue policy): Hệ thống có `MAX_PRIO = 140` mức ưu tiên (từ `0` đến `MAX_PRIO - 1`). Mỗi hàng đợi ưu tiên `prio` chỉ có một số lượng khe (`slot`) cố định

[1]Trên thực tế, Linux hỗ trợ cơ chế phản hồi (*feedback* – MLFQ) cho phép di chuyển tiến trình giữa các hàng đợi ưu tiên, nhưng chúng ta không hiện thực cơ chế feedback ở đây.

để sử dụng CPU trong một vòng duyệt, được tính theo công thức:

$$\text{slot} = \text{MAX_PRIO} - \text{prio}$$

Khi số `slot` của một hàng đợi đã dùng hết, hệ thống bắt buộc phải chuyển tài nguyên CPU sang cho tiến trình ở hàng đợi ưu tiên kế tiếp, để dành phần việc còn lại cho vòng duyệt sau (ngay cả khi hàng đợi hiện tại vẫn còn tiến trình đang chờ).

Ví dụ với `MAX_PRIO = 140`, `prio = 0..(MAX_PRIO - 1)`:

```
prio = 0         | 1                | .... | MAX_PRIO - 1
slot = MAX_PRIO | MAX_PRIO - 1    | .... | 1
```

Nhiệm vụ của bạn trong phần này là hoàn thiện các hàm sau:

- `enqueue()` và `dequeue()` (trong `src/queue.c`): Đưa một PCB mới vào hàng đợi và lấy PCB tiếp theo đến lượt ra khỏi hàng đợi (kèm hàm `purgequeue()` để xóa PCB khỏi `running_list`).
- `get_proc()` / `get_mlq_proc()` (trong `src/sched.c`): Lấy PCB của một tiến trình đang chờ từ hệ thống `mlq_ready_queue` theo đúng chính sách định mức `slot` ở trên.

Lưu ý: Vì luồng `loader` và luồng `scheduler/CPU` chạy đồng thời trên các thread khác nhau, có thể có nhiều hơn một đáp án đúng cho mỗi bài test. Ngoài ra, biến `run_queue` là phần mã cũ đã lỗi thời (*obsoleted*), chỉ giữ lại trong mã nguồn để tránh lỗi biên dịch chứ không cần dùng trong thuật toán MLQ.

Câu hỏi 3 (Trang 13 đề gốc)

Xét tác động của các chính sách chi tiết trong MLQ, lợi ích của từng chính sách là gì?

Góc giải thích dễ hiểu & Gợi ý làm phần Scheduler

- **Cách code `get_mlq_proc()`:** Duyệt `prio` từ `0` đến `MAX_PRIO - 1`. Nếu hàng đợi `mlq_ready_queue[prio]` không rỗng **VÀ** `slot[prio] > 0`, ta `dequeue()` lấy tiến trình ra, giảm `slot[prio]-` và trả về. Nếu duyệt hết cả 140 hàng đợi mà có tiến trình bị kẹt do `slot[prio] == 0`, ta reset lại toàn bộ `slot[i] = MAX_PRIO - i` rồi quét lại 1 lần nữa.
- **Gợi ý trả lời Câu hỏi 3:** Chính sách định mức `slot = MAX_PRIO - prio` kết hợp Round-Robin giúp **chống đói CPU (anti-starvation)**: tiến trình ưu tiên cao (`prio` nhỏ) vẫn được phục vụ nhiều nhất, nhưng không thể độc chiếm CPU mãi mãi vì khi hết `slot` phải nhường lượt cho các hàng đợi ưu tiên thấp hơn.

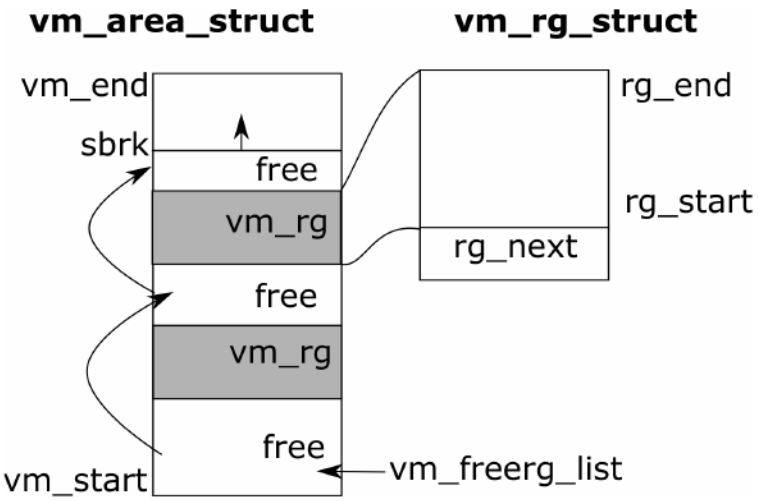
2.2 Quản lý Bộ nhớ (Memory Management)

2.2.1 Bố cục bộ nhớ của tiến trình (The process memory layout)

Không gian bộ nhớ ảo được tổ chức dưới dạng một cấu trúc ánh xạ bộ nhớ riêng cho mỗi PCB của tiến trình. Dưới góc nhìn của tiến trình, địa chỉ ảo bao gồm nhiều **khu vực nhớ ảo** (`vm_area_struct`) nằm liên tiếp nhau. Trong thực tế, mỗi khu vực này đóng vai trò như phân đoạn code, stack hoặc heap.

Khu vực bộ nhớ (Memory Area – `vm_area_struct`): Mỗi khu vực bộ nhớ trải dài liên tục trong đoạn `[vm_start, vm_end]`. Mặc dù không gian có thể mở rộng toàn bộ phạm vi, vùng thực

sự có thể sử dụng được giới hạn bởi đỉnh trỏ tại `sbrk`. Trong phạm vi giữa `vm_start` và `sbrk`, có nhiều vùng nhớ biến được quản lý bởi `struct vm_rg_struct` và các khoảng trống đã giải phóng được theo dõi bởi danh sách liên kết `vm_freerg_list`. Thông qua thiết kế này, việc cấp phát bộ nhớ vật lý thực sự chỉ diễn ra trong phạm vi khả dụng `[vm_start, sbrk]`, như minh họa ở Hình 4.



Hình 4: Cấu trúc của khu vực nhớ ảo (`vm_area`) và vùng nhớ biến (`vm_rg`)

```c
// Trich tu include/os-mm.h
/* Cau truc vung nho bien (Memory region struct) */
struct vm_rg_struct {
    int vmaid;
    addr_t rg_start;
    addr_t rg_end;
    struct vm_rg_struct *rg_next;
};

/* Cau truc khu vuc nho ao (Memory area struct) */
struct vm_area_struct {
    unsigned long vm_id;
    addr_t vm_start;
    addr_t vm_end;
    addr_t sbrk;
    struct mm_struct *vm_mm;
    struct vm_rg_struct *vm_freerg_list;
    struct vm_area_struct *vm_next;
};

/* Cau truc anh xa bo nho (Memory mapping struct) */
struct mm_struct {
#ifdef MM64
    addr_t *pgd;
    addr_t *p4d;
    addr_t *pud;
    addr_t *pmd;
    addr_t *pt;
#else
    uint32_t *pgd;
#endif
    struct vm_area_struct *mmap;
    struct vm_rg_struct symrgtbl[PAGING_MAX_SYMTBL_SZ];
    struct pgn_t *fifo_pgn;
```

```
35     struct kcache_pool_struct *kcpooltbl;
36 };
```

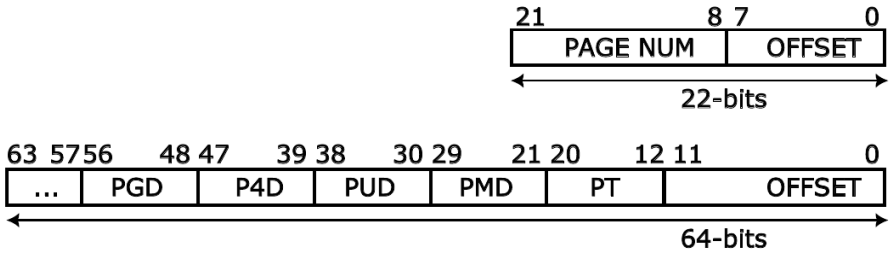
Vùng nhớ biến (Memory region): Như đã lưu ý ở Mục 1.3, các vùng nhớ này đóng vai trò như các biến trong mã nguồn chương trình. Chúng ta quản lý chúng bằng một mảng cố định `symrgtbl[PAGING_MAX_SYMTBL_SZ]` (tối đa 30 biến mỗi tiến trình), lưu điểm đầu `rg_start` và điểm cuối `rg_end` của từng biến.

Bố cục địa chỉ chuẩn tắc (Canonical address layout): Trong hệ thống 64-bit, phần cứng thường chỉ hiện thực địa chỉ 48-bit hoặc 57-bit thay vì toàn bộ 64-bit. Để tránh việc phần mềm lạm dụng các bit cao không dùng đến gây mất nhất quán, kiến trúc hiện đại áp dụng quy tắc *địa chỉ chuẩn tắc (canonical address)*: tất cả các bit cao không sử dụng phải được cố định toàn `0` (thường dùng cho user space) hoặc toàn `1` (thường dùng cho kernel space). Trong phần Phụ lục (trang 29–32), đề bài cung cấp bảng bố cục địa chỉ của Linux, macOS và Windows NTKernel để sinh viên tham khảo.

Các thao tác bộ nhớ cơ bản trong vùng nhớ nhân (Kernel memory): Trong học kỳ này, chúng ta không đi sâu vào thuật toán quản lý bộ nhớ nhân phức tạp mà giữ nó đơn giản như một chiến lược cấp phát vùng nhớ liên tục. Tuy nhiên, hệ thống vẫn yêu cầu **hạn chế quyền truy cập bộ nhớ đặc quyền giữa User space và Kernel space**: chương trình người dùng chỉ được dùng `read`/`write` trên địa chỉ user space; việc truy cập thẳng vào địa chỉ kernel từ user space bị cấm và phải thực hiện truyền dữ liệu qua `copy_from_user` và `copy_to_user`.

2.2.2 Ánh xạ bộ nhớ ảo trong mỗi tiến trình (Virtual memory mapping in each process)

Ánh xạ bộ nhớ được đại diện bởi `struct mm_struct`, chứa danh sách các khu vực nhớ `mmap`, bảng ký hiệu `symrgtbl`, danh sách trang FIFO `fifo_pgn` phục vụ thay thế trang, và đặc biệt là hệ thống bảng trang (`pgd`, `p4d`, `pud`, `pmd`, `pt`).



Hình 5: Sơ đồ địa chỉ CPU 22-bit/32-bit và 64-bit 5 cấp (CPU address scheme)

Sơ đồ địa chỉ CPU 32-bit: Chia địa chỉ thành **Số hiệu trang (Page number** – p**)** làm chỉ số tra bảng trang và **Độ lệch trang (Page offset** – d**)** kết hợp với địa chỉ khung trang cơ sở để tạo thành địa chỉ vật lý gửi tới MMU. Bảng 1 liệt kê các cấu hình bus địa chỉ 32-bit.

CPU bus	PAGE sz	PAGE bit	No pg entry	Entry sz	PAGE TBL	OFFSET bit	PGT mem	M
20	256B	12	~4000	4byte	16KB	8	2MB	1M
22	**256B**	**14**	**~16000**	**4byte**	**64KB**	**8**	**8MB**	**1M**
22	512B	13	~8000	4byte	32KB	9	4MB	1M
22	512B	13	~8000	4byte	32KB	9	4MB	12
16	512B	8	256	4byte	1kB	9	128K	12

Bảng 1: Các cấu hình bus địa chỉ CPU khác nhau (Various CPU address bus configurations)

Sơ đồ địa chỉ CPU 64-bit (5-level paging scheme): Áp dụng phân trang 5 cấp trên CPU

64-bit, mở rộng không gian địa chỉ ảo lên tới 128 Petabyte (PiB) với kích thước trang **4KB (4096 byte)**:

- **Cấp 5 – Page Global Directory (PGD):** bit 56–48 (9 bit)
- **Cấp 4 – Page Level 4 Directory (P4D):** bit 47–39 (9 bit)
- **Cấp 3 – Page Upper Directory (PUD):** bit 38–30 (9 bit)
- **Cấp 2 – Page Middle Directory (PMD):** bit 29–21 (9 bit)
- **Cấp 1 – Page Table (PT):** bit 20–12 (9 bit)
- **Độ lệch trang – Page offset (OFFSET):** bit 11–0 (12 bit $\Rightarrow 2^{12} = 4096$ B = 4 KB)

Bit	63–57	56–48	47–39	38–30	29–21	20–12	11–0
Cấp phân trang	—	L5	L4	L3	L2	L1	OFFSET
Thư mục trang	—	PGD	P4D	PUD	PMD	PT	OFFSET
Kích thước vùng nhớ	—	128PiB	256TB	512GB	1GB	2MB	4KB

Bảng 2: Sơ đồ phân trang 5 cấp trên CPU 64-bit (CPU 64-bit scheme)

Ví dụ tách chỉ số 5 cấp từ địa chỉ ảo `0x39ffe1d9c9000`:

- Cấp 1 (PT): Index = (`0x39ffe1d9c9000` $\gg$ 12) & `0x1ff` = `0x1c9`
- Cấp 2 (PMD): Index = (`0x39ffe1d9c9000` $\gg$ 21) & `0x1ff` = `0x0ec`
- Cấp 3 (PUD): Index = (`0x39ffe1d9c9000` $\gg$ 30) & `0x1ff` = `0x1f8`
- Cấp 4 (P4D): Index = (`0x39ffe1d9c9000` $\gg$ 39) & `0x1ff` = `0x13f`
- Cấp 5 (PGD): Index = (`0x39ffe1d9c9000` $\gg$ 48) & `0x1ff` = `0x03`

Tối ưu kích thước bảng trang đa cấp: Vì bảng trang cho địa chỉ 64-bit rất lớn nếu cấp phát toàn bộ, có nhiều chiến lược để tối ưu như: *Cấp phát thưa kết hợp băm (Sparse allocation with hashing)* hoặc *Cấp phát động theo nhu cầu (Demand allocation)* – chỉ tạo bảng trang cho những vùng địa chỉ thực sự đang sử dụng. Toàn bộ cấu trúc hỗ trợ VM nằm ở `mm-vm.c`, `mm.c` và hỗ trợ địa chỉ 64-bit nằm ở `mm64.c`.

Câu hỏi 4 (Trang 17 đề gốc)

Động lực chính của việc kết hợp phân đoạn (*segmentation*) với phân trang (*paging*) trong quản lý bộ nhớ là gì? Cách tiếp cận lai (*hybrid*) này giải quyết những hạn chế cố hữu khi chỉ sử dụng riêng lẻ từng kỹ thuật như thế nào?

Gợi ý trả lời Câu hỏi 4 cho Báo cáo

- **Hạn chế riêng lẻ:** *Phân đoạn thuần túy* chia bộ nhớ theo góc nhìn logic (code, data, heap, stack) dễ bảo vệ/chia sẻ nhưng gây **phân mảnh ngoại vi (external fragmentation)** vì cần vùng RAM vật lý liên tục lớn. Ngược lại, *Phân trang thuần túy* triệt tiêu phân mảnh ngoại vi bằng cách chia thành các frame cố định nhưng lại mất đi ranh giới ngữ nghĩa logic giữa các vùng nhớ của chương trình.
- **Kết hợp (Segmentation with Paging):** Như trong bài tập này (`vm_area_struct` làm phân đoạn logic + bảng trang `pgd..pt` ánh xạ xuống frame vật lý), tiến trình vẫn quản lý các phân đoạn liên tục về mặt địa chỉ ảo và kiểm soát quyền truy cập dễ dàng, trong khi ở tầng dưới mỗi phân đoạn được chia thành các trang 4KB nằm rải rác trên RAM vật lý, loại bỏ hoàn toàn phân mảnh ngoại vi.

2.2.3 Bộ nhớ vật lý của hệ thống (The system's physical memory)

Tất cả các tiến trình có ánh xạ bộ nhớ ảo riêng biệt nhưng đều trỏ về các thiết bị vật lý dùng chung của toàn hệ thống trong `mm-memphy.c`. Có hai loại thiết bị vật lý: **RAM** và **SWAP**:

- **Thiết bị RAM (`mram`):** Thuộc phân hệ bộ nhớ chính, được CPU truy cập trực tiếp qua bus địa chỉ để đọc/ghi bằng lệnh CPU.
- **Thiết bị SWAP (`mswp`):** Là thiết bị lưu trữ thứ cấp; CPU không thể đọc/ghi trực tiếp dữ liệu trên SWAP mà phải di chuyển trang dữ liệu đó vào RAM trước (*swap in*). Hệ thống hỗ trợ **1 thiết bị RAM** và **tối đa 4 thiết bị SWAP**.

```c
// Trich tu include/os-mm.h
struct framephy_struct {
    addr_t fpn;
    struct framephy_struct *fp_next;
    struct mm_struct* owner;
};

struct memphy_struct {
    /* Truong du lieu co ban va kich thuoc */
    BYTE *storage;
    int maxsz;
    /* Truong danh cho thiet bi truy cap tuan tu */
    int rdmflg;
    int cursor;
    /* Cau truc quan ly danh sach khung trang */
    struct framephy_struct *free_fp_list;
    struct framephy_struct *used_fp_list;
};
```

Câu hỏi 5 (Trang 18 đề gốc)

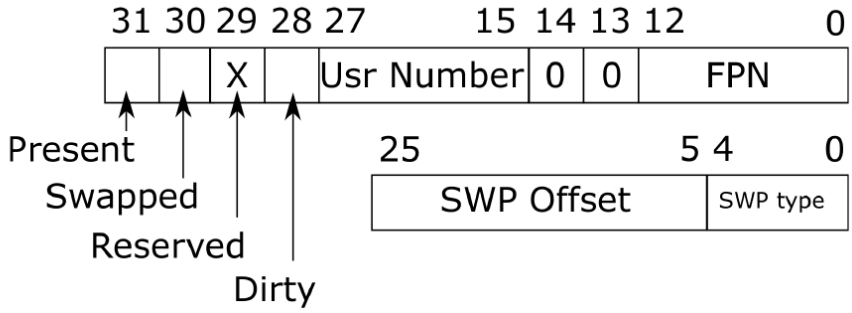
Lợi ích của việc mở rộng phân trang phân cấp (*hierarchical paging*) lên N cấp là gì?

Gợi ý trả lời Câu hỏi 5 cho Báo cáo

Nếu dùng bảng trang 1 cấp cho địa chỉ 57-bit (trang 4KB), bảng trang sẽ cần 2^{45} phần tử ($\approx$ 256 TB RAM cho mỗi tiến trình!) và phải nằm liên tục trong bộ nhớ. Chia thành N cấp (như 5 cấp PGD $\rightarrow$ PT, mỗi bảng con chỉ 512 phần tử = 4KB vừa khít 1 trang vật lý) giúp: (1) không cần cấp phát một vùng nhớ khổng lồ liên tục cho bảng trang, và (2) chỉ cần cấp phát các bảng con ở cấp thấp cho những vùng địa chỉ mà tiến trình thực sự chạm tới, tiết kiệm hàng triệu lần dung lượng RAM.

2.2.4 Cơ chế dịch địa chỉ dựa trên phân trang (Paging-based address translation scheme)

Mỗi phần tử bảng trang (**Page Table Entry – PTE**) chứa một giá trị 32-bit (hoặc lưu trong `addr_t`) cho mỗi trang ảo, giúp xác định khung trang vật lý (`FPN`) hoặc vị trí trên SWAP mà trang ảo đó đang được ánh xạ tới, như minh họa ở Hình 6.



Hình 6: Định dạng của một phần tử bảng trang (Page Table Entry Format)

```
* Bits 0-12  : so hieu khung trang vat ly (FPN - page frame number) neu
   trang dang o RAM (present)
* Bits 13-14 : bang 0 neu trang dang o RAM (present)
* Bits 15-27 : so danh dau do nguoi dung dinh nghia (user-defined
   numbering) neu present
* Bits 0-4   : loai thiet bi swap (swap type) neu trang bi day ra SWAP (
   swapped)
* Bits 5-25  : do lech tren swap (swap offset / swpfpn) neu trang bi
   swapped
* Bit  28    : bit danh dau trang da bi ghi de (dirty)
* Bit  29    : danh rieng (reserved)
* Bit  30    : bit danh dau trang dang o SWAP (swapped)
* Bit  31    : bit danh dau trang hop le/hien dien (presented)
```

Hoán đổi bộ nhớ (Memory swapping): Khi RAM đầy, hệ thống thực hiện **Swapping out** (chuyển nội dung một khung trang từ `MEMRAM` sang `MEMSWAP` để giải phóng khung RAM) và **Swapping in** (sao chép nội dung trang cần dùng từ `MEMSWAP` trở lại `MEMRAM`).

Các thao tác bộ nhớ cơ bản trong hệ thống phân trang:

- `ALLOC`: Tìm vùng trống trong `vm_freerg_list`. Nếu không có vùng trống đủ lớn, hệ thống gọi lời gọi hệ thống MMU (`SYSCALL 17` với `SYSMEM_INC_OP`) để nâng rào cản `sbrk`, xin cấp phát các khung trang vật lý mới và lập ánh xạ vào bảng trang.
- `FREE`: Thu hồi vùng nhớ ứng với chỉ số biến `rgid`. Vì không thể trả ngay khung trang vật lý về RAM mà không làm ảnh hưởng các biến khác nằm chung trang, thư viện `libmem` đưa vùng

địa chỉ ảo vừa giải phóng vào danh sách vùng trống `vm_freerg_list` để tái sử dụng cho các lệnh `ALLOC` sau.

- `READ/WRITE`: Đòi hỏi trang chứa địa chỉ cần truy cập phải đang nằm trong RAM (`Present`). Nếu trang đang nằm ở thiết bị `MEMSWAP`, hệ thống phải chọn một trang nạn nhân (`find_victim_page`), đẩy trang nạn nhân ra SWAP (`SYSMEM_SWP_OP`), nạp trang cần dùng vào RAM, cập nhật PTE rồi mới thực hiện đọc/ghi (`SYSMEM_IO_READ` / `SYSMEM_IO_WRITE`).

Câu hỏi 6 (Trang 19 đề gốc)

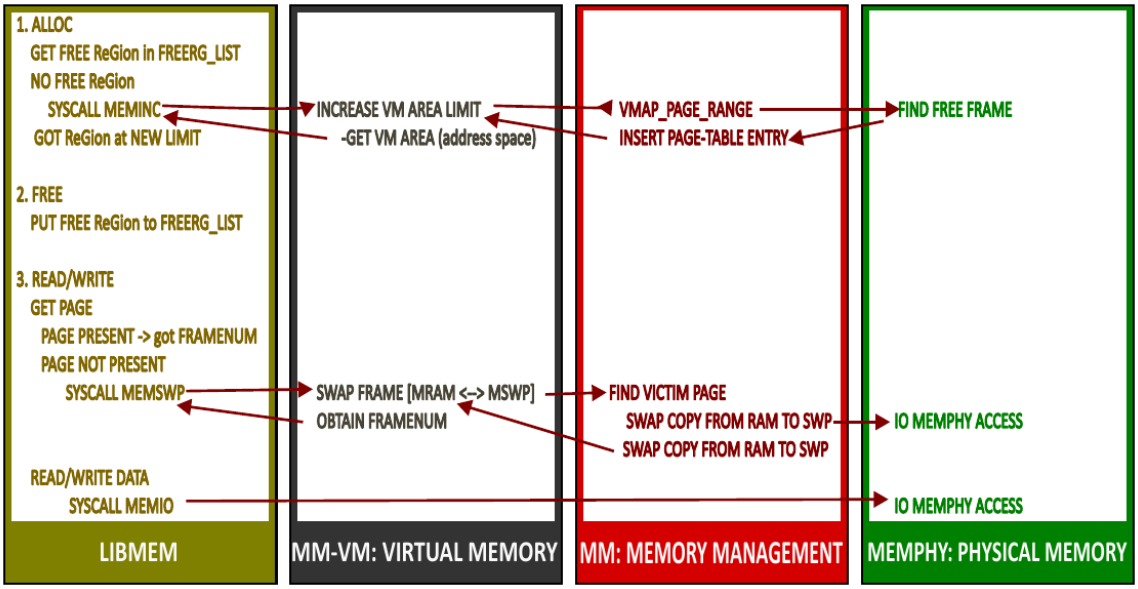
Ưu điểm và nhược điểm của cơ chế phân trang (*paging*) so với cấp phát bộ nhớ liên tục (*contiguous memory allocation*) là gì?

Gợi ý trả lời Câu hỏi 6 cho Báo cáo

- **Cấp phát bộ nhớ liên tục:** *Ưu điểm:* Dịch địa chỉ cực nhanh (chỉ cần thanh ghi `base + limit`), không tốn bộ nhớ lưu bảng trang. *Nhược điểm:* Bị phân mảnh ngoại vi nghiêm trọng, khó mở rộng kích thước tiến trình động và khó swap từng phần.
- **Phân trang (Paging):** *Ưu điểm:* Loại bỏ hoàn toàn phân mảnh ngoại vi, cấp phát linh hoạt từng khung 4KB rải rác, dễ dàng hoán đổi từng trang ra SWAP và chia sẻ bộ nhớ. *Nhược điểm:* Tốn bộ nhớ lưu bảng trang, tốn thêm chu kỳ truy cập bộ nhớ khi tra bảng trang đa cấp (nếu trượt TLB) và có thể bị phân mảnh nội vi (*internal fragmentation*) ở trang cuối cùng.

2.2.5 Tổng kết các phần hiện thực hướng phân trang (Wrapping-up all paging-oriented implementations)

Để thực hiện các thao tác trên, các phân hệ `LIBMEM`, `MM-VM`, `MM/MM64` và `MEMPHY` phối hợp chặt chẽ với nhau theo sơ đồ ở Hình 7.



Hình 7: Sự phối hợp giữa các phân hệ quản lý bộ nhớ (Memory system modules)

Kiểm soát cấu hình thông qua định nghĩa hằng số trong `include/os-cfg.h`:

```
// Trich tu include/os-cfg.h
#define MLQ_SCHED 1
#define MAX_PRIO 140

#define MM_PAGING
//#define MM_FIXED_MEMSZ
//#define VMDBG 1
//#define MMDBG 1
#define IODUMP 1
#define PAGETBL_DUMP 1

#define MM64 1
//#undef MM64
```

Cấu hình khai báo tường minh kích thước bộ nhớ: Khi đóng chú thích (comment out) hằng số `#define MM_FIXED_MEMSZ`, chương trình mô phỏng sẽ đọc thêm dòng cấu hình kích thước bộ nhớ vật lý từ file `input/`: 1 thiết bị `MEMRAM` và tối đa 4 thiết bị `MEMSWP` (phải có ít nhất 1 RAM $>$ 0 và 1 SWAP $>$ 0, các SWAP còn lại có thể bằng 0):

```
[time slice] [N = So luong CPU] [M = So luong tien trinh]
[RAM_SZ] [SWP_SZ_0] [SWP_SZ_1] [SWP_SZ_2] [SWP_SZ_3]
[time 0] [path 0] [priority 0]
...
```

2.3 Bộ đồng xử lý (Co-processors)

2.3.1 Hoạt động bất đồng bộ và địa chỉ bus (Asynchronous operation and bus address)

Hoạt động bất đồng bộ (*asynchronous operation*) được minh họa trong Hình 8: CPU khởi tạo một yêu cầu I/O rồi tiếp tục thực thi các lệnh khác thay vì đứng chờ I/O hoàn tất. Khi bộ điều khiển I/O hoàn thành thao tác, nó phát tín hiệu ngắt (*interrupt* – Bước 4). CPU tạm dừng tác vụ hiện tại, chạy trình xử lý ngắt (*interrupt handler* / *callback*) để xử lý kết quả I/O rồi quay lại công việc đang dang dở.

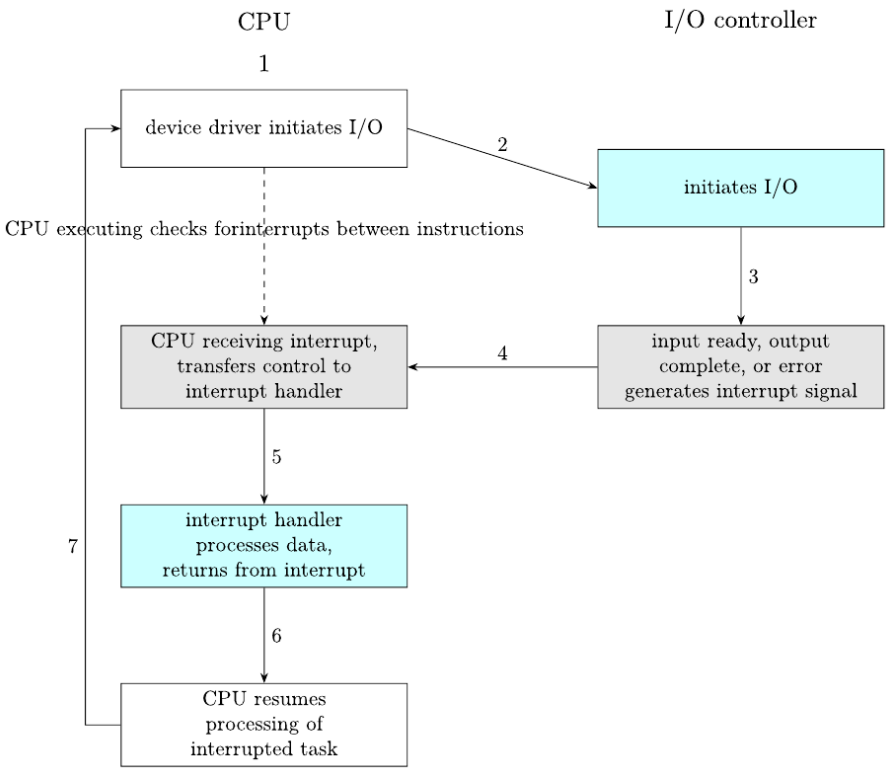
Hình 9 trình bày cách một máy tính hiện đại vận hành: nhiều lõi CPU liên tục nạp lệnh và dữ liệu từ bộ nhớ qua bộ nhớ đệm (cache) để thực thi, trong khi các thiết bị ngoại vi/đồng xử lý thực hiện I/O bất đồng bộ và trao đổi dữ liệu trực tiếp với bộ nhớ thông qua **địa chỉ bus (bus address)**.

2.3.2 Truy cập bộ nhớ trực tiếp – DMA (Direct Memory Access)

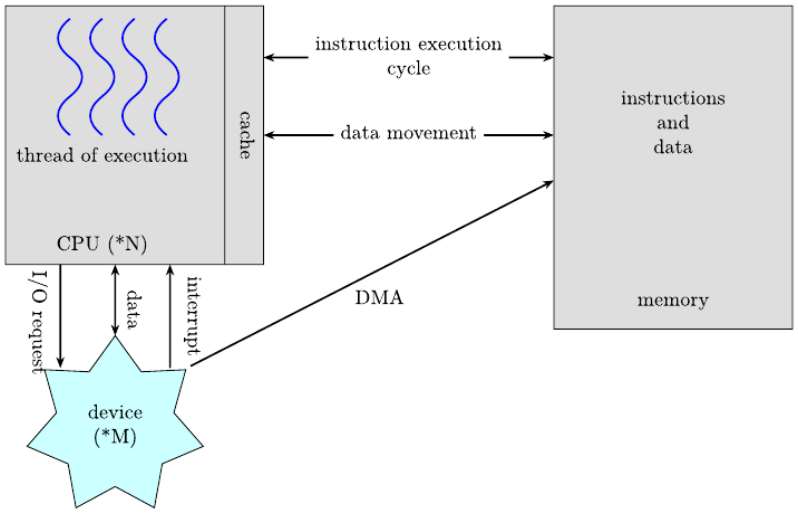
DMA là một đơn vị phần cứng chuyên dụng quản lý việc truyền dữ liệu giữa các vùng nhớ máy tính mà không cần CPU đứng canh chép từng byte:

- **Vấn đề khi không có DMA (Programmed I/O):** CPU phải tự đọc thủ công từng byte dữ liệu từ nguồn vào thanh ghi rồi ghi ngược lại vào RAM đích, làm lãng phí chu kỳ CPU.
- **Giải pháp dùng bộ đồng xử lý DMA:** CPU chỉ cần báo cho bộ điều khiển DMA biết địa chỉ nguồn (`source`), địa chỉ đích (`destination`), và kích thước cần chép (`length`), sau đó quay lại làm việc khác. Bộ điều khiển DMA sẽ nắm quyền điều khiển bus hệ thống, tự di chuyển dữ liệu trực tiếp và chỉ báo cho CPU (qua callback/ngắt) khi toàn bộ quá trình truyền hoàn tất.

Trong thiết lập hiện tại của mô phỏng phần cứng DMA (`dma_top` bằng Verilator), chúng ta sử dụng:



Hình 8: Cơ chế vào/ra hướng ngắt bất đồng bộ (Interrupt-driven I/O)



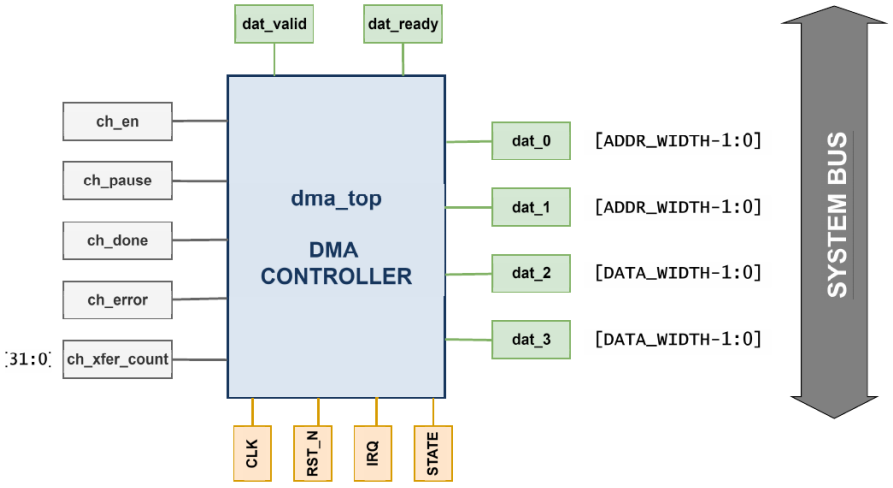
Hình 9: Cách thức hoạt động của máy tính hiện đại (Modern computer works)

- `ADDR_WIDTH`: 64 bits
- `DATA_WIDTH`: 32 bits

Do sự phức tạp của việc lệch thứ tự byte (*endianness*), sinh viên được khuyến khích xử lý dữ liệu theo từng byte ở tầng phần mềm nếu cần, mặc dù giao diện phần cứng hỗ trợ truyền 32-bit mỗi lần đọc/ghi.

Tên tín hiệu	Chiều	Độ rộng	Mô tả chức năng
`CLK`	Input	1	Tín hiệu xung nhịp đồng hồ (Clock signal).
`RST_N`	Input	1	Tín hiệu khởi tạo lại (Reset tích cực mức thấp).
`IRQ`	Output	1	Đường tín hiệu yêu cầu ngắt gửi tới bộ xử lý.
`STATE`	Output	2	Trạng thái hoạt động hiện tại của module DMA.
`ch_en`	Input	1	Tín hiệu kích hoạt kênh (Channel enable) để bắt đầu truyền DMA.
`ch_pause`	Input	1	Tín hiệu tạm dừng kênh truyền.
`ch_done`	Output	1	Được bật lên mức cao khi kênh DMA hoàn tất truyền dữ liệu.
`ch_error`	Output	1	Được bật lên nếu xảy ra lỗi trong quá trình truyền.
`ch_xfer_count`	Output	32	Tổng số đơn vị dữ liệu đã truyền thành công.
`dat_valid`	Input	1	Báo hiệu dữ liệu cấu hình đầu vào trên các đường `dat_*` là hợp lệ.
`dat_0`	In/Out	ADDR_WIDTH	Đường địa chỉ đầu vào 0 (Địa chỉ nguồn – Source offset).
`dat_1`	In/Out	ADDR_WIDTH	Đường địa chỉ đầu vào 1 (Địa chỉ đích – Destination offset).
`dat_2`	In/Out	DATA_WIDTH	Đường dữ liệu đầu vào 2 (Độ dài cần chép – Length).
`dat_3`	In/Out	DATA_WIDTH	Đường dữ liệu đầu vào 3 (Dự phòng / Tham số phụ).
`dat_ready`	Output	1	Tín hiệu báo module DMA đã sẵn sàng nhận lệnh.

Bảng 3: Mô tả các chân tín hiệu của module phần cứng DMA (DMA module pin description)



Hình 10: Sơ đồ khối bộ đồng xử lý DMA (DMA co-processor)

2.3.3 Bộ xử lý đồ họa – GPU & Các công nghệ tăng tốc I/O

- **GPU (Graphic Processor Unit):** Cũng hoạt động bất đồng bộ như các bộ đồng xử lý khác nhưng được thiết kế cho tính toán song song cực lớn (hàng trăm nghìn phép toán ma trận/vector cùng lúc).
- **Các công nghệ tăng tốc I/O (I/O acceleration technologies):**

- *Memory operation offloading:* Chuyển giao việc sao chép bộ nhớ sang bộ nhớ cho engine chipset chuyên dụng để giải phóng chu kỳ CPU.
- *Traffic and network optimization:* Đưa tải dữ liệu mạng thẳng vào bộ nhớ ứng dụng mà không bắt CPU xử lý từng gói.
- *Extended signaled interrupt:* Phân phối đều ngắt I/O qua nhiều engine chipset.
- *Low-latency interrupt:* Tự động điều chỉnh tần suất ngắt theo mức độ khẩn cấp.
- *Streaming Accelerator & Pass through I/O:* Tăng tốc luồng dữ liệu và tối đa hóa thông lượng truyền song song.

2.4 Kết hợp tất cả lại & Đồng bộ hóa (Put It All Together)

Cuối cùng, chúng ta kết hợp bộ lập lịch, quản lý bộ nhớ và bộ đồng xử lý để tạo thành một Hệ điều hành hoàn chỉnh. Hình 11 thể hiện tổ chức tổng thể của toàn bộ phân hệ quản lý bộ nhớ trong HĐH.

Nhiệm vụ cuối cùng là Đồng bộ hóa (Synchronization): Vì hệ điều hành chạy trên nhiều bộ vi xử lý (nhiều luồng CPU, luồng Loader, luồng Co-processor chạy song song), các tài nguyên dùng chung (*shared resources* như hàng đợi `mlq_ready_queue`, `running_list`, thiết bị vật lý `mram`/`mswp`, vùng nhớ nhân `krnl`, và phần cứng `DMA`) có thể bị truy cập đồng thời bởi nhiều tiến trình cùng lúc. Nhiệm vụ của bạn là xác định các tài nguyên dùng chung này và sử dụng cơ chế khóa (`pthread_mutex_lock` / `pthread_mutex_unlock`) để bảo vệ chúng.

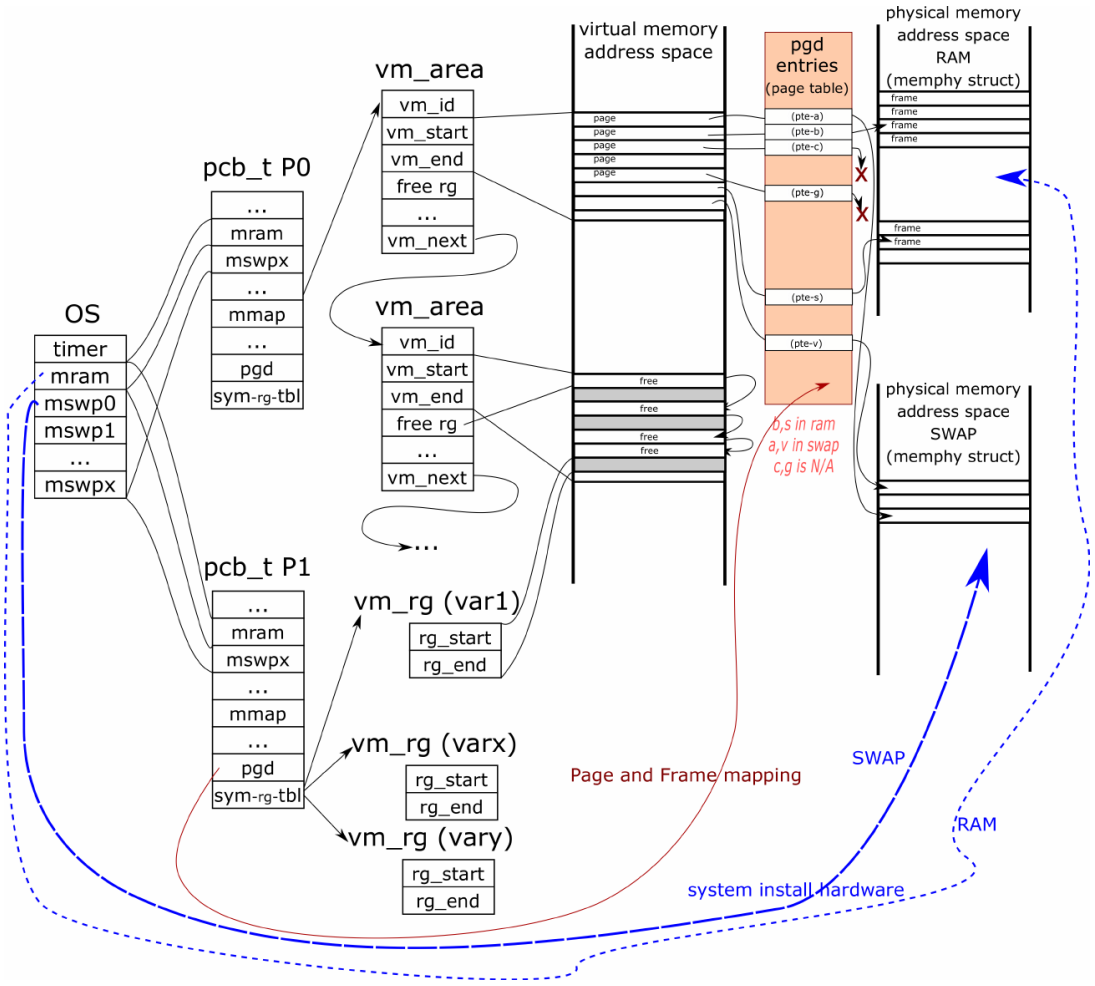
Câu hỏi 7 (Trang 25 đề gốc)

Điều gì sẽ xảy ra nếu việc đồng bộ hóa (*synchronization*) không được xử lý trong Hệ điều hành Đơn giản của bạn? Hãy minh họa vấn đề của hệ điều hành (thông qua kết quả đầu ra của bài tập) bằng ví dụ cụ thể nếu có trong các thao tác bộ nhớ nhân được thêm vào.

Gợi ý trả lời Câu hỏi 7 cho Báo cáo

Nếu không có mutex lock (`queue_lock`, `mmvm_lock`, `kmm_lock`), hiện tượng **điều kiện tranh chấp (Race Condition)** sẽ xảy ra khi chạy đa CPU (như bài test `os_1_mlq_paging` hoặc `os_2_mlq_paging` với 4 CPU):

1. Hai CPU cùng gọi `get_proc()` một lúc sẽ lấy trùng cùng một tiến trình (2 CPU chạy chung 1 PID!) hoặc làm hỏng biến đếm `q->size`.
2. Hai tiến trình trên 2 CPU cùng gọi `MEMPHY_get_freefp()` hoặc `kmalloc` cùng lúc sẽ nhận được **cùng một khung trang vật lý** `fpn` hoặc cùng một vùng nhớ nhân, dẫn đến ghi đè dữ liệu của nhau hoặc lỗi `Segmentation Fault (double free)` khi giải phóng danh sách liên kết.



Hình 11: Sơ đồ tổng thể hoạt động liên quan đến bộ nhớ ảo trong bài tập lớn (The operation related to virtual memory in the assignment)

3 Nộp bài và Đánh giá (Submission)

3.1 Mã nguồn (Source code)

Yêu cầu: Bạn phải viết mã nguồn cho các lời gọi hệ thống và các phân hệ tuân theo quy chuẩn viết code C của GNU (https://www.gnu.org/prep/standards/html_node/Writing-C.html).

3.2 Tóm tắt các Yêu cầu Hiện thực (Requirements)

1. **Bộ lập lịch (Scheduler):** Hiện thực bộ lập lịch áp dụng chính sách MLQ như mô tả ở Mục 2.1.
2. **Quản lý bộ nhớ với không gian User/Kernel tách biệt (Memory Management with separable user/kernel space):** Hiện thực phân hệ phân trang.

   ****Lưu ý quan trọng**:** Không gian bộ nhớ được tách biệt giữa không gian người dùng (*user space*) và không gian nhân (*kernel space*). Việc truy cập trực tiếp từ user space thông qua con trỏ `pcb_t` của tiến trình sang kernel là **không được phép**; khi đi qua lời gọi hệ thống (`syscall`), chỉ được truyền `struct krnl_t` và `pid`, sau đó nhân phải tự tra cứu lại PCB từ `krnl`.
3. **Phân trang đa cấp (Multi-level paging):** Hiện thực sơ đồ địa chỉ dài 64-bit với cơ chế phân trang đa cấp lý thuyết (5 cấp).
   - *Thay thế trang tùy chọn (Optional page replacement):* Làm việc với sơ đồ địa chỉ dài 64-bit là một tác vụ phức tạp. Do đó, yêu cầu về thay thế trang được định nghĩa bởi giảng viên của bạn. Sinh viên được khuyến khích đưa ra quyết định thiết kế thông qua việc so sánh cẩn thận các chiến lược. Trong báo cáo và kết quả đầu ra, sinh viên cần trình bày thống kê về số lần truy cập bộ nhớ cũng như kích thước lưu trữ của bảng trang đa cấp, đồng thời giải thích lợi ích từ thiết kế của mình.
4. **Bộ đồng xử lý (Co-processor):** Hiện thực cơ chế tương tác bất đồng bộ giữa CPU và bộ đồng xử lý:
   - **Tất cả sinh viên (All students):** Bộ máy đồng xử lý DMA đã được cung cấp sẵn. Phần Phụ lục (trang 33) có đính kèm một nguyên mẫu phần mềm (*software prototype*) của DMA để sinh viên có thể dùng thử nghiệm sơ bộ. Tuy nhiên, khi nộp bài và thuyết trình đồ án nhóm, **bắt buộc phải sử dụng bộ máy chip DMA thực tế** được hỗ trợ bởi nền tảng phần cứng thông qua `verilator` (`Vdma_top`).
   - **Sinh viên tài năng (Talent students – Không cộng điểm, chỉ để thử thách bản thân):** Được khuyến khích tự thiết kế thêm một chip đồng xử lý mới (GPU) hỗ trợ thư viện CUDA và OpenVINO cho tính toán ma trận/vector.
5. **Bảng câu hỏi (Questionnaire):** Sinh viên cần trả lời toàn bộ 7 câu hỏi (*Question*) nằm rải rác trong bản mô tả bài tập lớn này vào trong Báo cáo.

3.3 Báo cáo (Report)

Viết một bản báo cáo trả lời các câu hỏi trong phần hiện thực và giải thích kết quả chạy kiểm thử ở từng phần:

- **Lập lịch (Scheduling):** Vẽ biểu đồ **Gantt** mô tả cách các tiến trình được thực thi bởi CPU theo thời gian.

- **Quản lý bộ nhớ (Memory management):** Trình bày trạng thái cấp phát bộ nhớ trong các phân đoạn dữ liệu. Bạn phải đảm bảo và chứng minh rằng giao tiếp giữa kernel space và user space thông qua system call sử dụng cơ chế **truyền PID (PID passing)** chứ không truyền trực tiếp con trỏ PCB (*not direct PCB passing*).
- **Phân trang đa cấp (Multi-level paging):** Trình bày sơ đồ dịch địa chỉ phân trang đa cấp 64-bit.
- **Tổng thể (Overall):** Sinh viên tự đưa ra cách trình bày và giải thích hợp lý đối với các kết quả mô phỏng thu được.

Sau khi hoàn thành bài tập lớn, hãy chép file báo cáo (PDF) vào thư mục mã nguồn, nén toàn bộ thư mục thành một file duy nhất có tên `assignment_STUDENTID.zip` và nộp lên hệ thống LMS.

3.4 Thang điểm Đánh giá (Grading)

Bài tập lớn được thực hiện theo nhóm từ **4 đến 5 sinh viên**. Điểm tổng kết của nhóm được quyết định bởi các thành phần sau (Tổng: **10 điểm**):

- **Chấm thực hành / Vấn đáp (Demonstration – 7 điểm):**
  - **Lập lịch (Scheduling): 2 điểm**
  - **MMU và không gian User/Kernel: 2 điểm** *(Phần công việc địa chỉ bộ nhớ này có phần giao thoa với dải địa chỉ của phần tiếp theo, nhưng được tính điểm kết hợp cùng phần sau).*
  - **Bộ đồng xử lý (Co-processor / DMA): 3 điểm**
    - **1 điểm:** Hoạt động bất đồng bộ (*asynchronous operation*).
    - **2 điểm:** Định địa chỉ Bus (*bus addressing*) – trong đó 1 điểm cho việc CPU truy cập bộ nhớ thông qua MMU, và 1 điểm cho việc bộ đồng xử lý DMA truy cập vào bộ nhớ.
- **Báo cáo (Report – 3 điểm)**

3.5 Quy tắc đạo đức & Bản quyền (Code of ethics)

Các giảng viên tham gia phát triển mã nguồn giữ toàn bộ bản quyền đối với mã nguồn dự án. Cấp phép Mã nguồn: Tác giả cấp quyền cá nhân cho sinh viên sử dụng và chỉnh sửa Mã nguồn được cấp phép với mục đích duy nhất là học tập trong thời gian theo học môn CO2018 tại Trường ĐH Bách Khoa TP.HCM (HCMUT).

Lịch sử Phiên bản (Revision History)

Phiên bản	Thời gian	Tác giả	Mô tả thay đổi
1.0	01.2019	pdnguyen, M.T. CHUNG, H.D. NGUYEN	Khởi tạo CPU, lập lịch, bộ nhớ cơ bả
1.1	09.2022	pdnguyen	Thêm lập lịch CPU hàng đợi đa mức
2.0	03.2023	pdnguyen	Khởi tạo khung phân trang bộ nhớ (MI
2.1	10.2023	pdnguyen	Thêm cơ chế thay thế trang (Page Rep
2.2	03.2024	pdnguyen	Thêm bộ đệm dịch địa chỉ TLB
2.3	10.2024	pdnguyen	Thêm phân đoạn Heap
3.0	03.2025	pdnguyen	Thêm lời gọi hệ thống (System call)
4.0	10.2025	pdnguyen	Thêm địa chỉ 64-bit (5-level paging)
4.1	03.2026	pdnguyen	Thêm địa chỉ chuẩn tắc (Canonical ac
5.0	**10.2026**	**pdnguyen**	**Thêm bộ đồng xử lý (Co-pro DMA RTL)**

A Phụ lục: Bố cục Địa chỉ Linux (Linux Address Layout)

Bố cục được trình bày theo thứ tự từ địa chỉ thấp đến địa chỉ cao (hệ thống 64-bit phân trang 5 cấp):

Địa chỉ đầu (hex)	Offset	Địa chỉ cuối (hex)	Kích thước	Mô tả khu vực nhớ ảo (VM area)
0000000000000000	0	00ffffffffffffff	64 PB	Bộ nhớ ảo không gian người dùng (*user-space*), riêng biệt cho mỗi `mm`
0100000000000000	+64 PB	feffffffffffffff	~16K PB	Khoảng trống khổng lồ chứa các địa chỉ ảo không chuẩn tắc (*non-canonical hole*)
Bộ nhớ không gian nhân (Kernel-space memory), được chia sẻ giữa mọi tiến trình				
ff00000000000000	-64 PB	ff0fffffffffffff	4 PB	Khoảng trống bảo vệ (*guard hole*) / dành riêng cho hypervisor
ff10000000000000	-60 PB	ff10ffffffffffff	0.25 PB	Ánh xạ lại LDT cho cơ chế cô lập bảng trang PTI
ff11000000000000	-59.75 PB	ff90ffffffffffff	32 PB	Ánh xạ trực tiếp toàn bộ bộ nhớ vật lý (`page_offset_base`)
ff91000000000000	-27.75 PB	ff9fffffffffffff	3.75 PB	Khoảng trống chưa sử dụng (*unused hole*)
ffa0000000000000	-24 PB	ffd1ffffffffffff	12.5 PB	Không gian `vmalloc/ioremap` (`vmalloc_base`)
ffd2000000000000	-11.5 PB	ffd3ffffffffffff	0.5 PB	Khoảng trống chưa sử dụng
ffd4000000000000	-11 PB	ffd5ffffffffffff	0.5 PB	`vmemmap` (mảng các `struct page`)
ffd6000000000000	-10.5 PB	ffdeffffffffffff	2.25 PB	Khoảng trống chưa sử dụng
ffdf000000000000	-8.25 PB	fffffbffffffffff	~8 PB	Bộ nhớ bóng KASAN (*KASAN shadow memory*)
Bố cục giống hệt với 47LA (phân trang 4 cấp) từ đây trở đi				
ffffffc000000000	-4 TB	fffffdffffffffff	2 TB	Khoảng trống chưa sử dụng (`vaddr_end` cho KASLR)
fffffe0000000000	-2 TB	fffffe7fffffffff	0.5 TB	Ánh xạ vùng `cpu_entry_area`
fffffe8000000000	-1.5 TB	fffffeffffffffff	0.5 TB	Khoảng trống chưa sử dụng
ffffff0000000000	-1 TB	ffffff7fffffffff	0.5 TB	Ngăn xếp sửa lỗi `%esp fixup stacks`
ffffff8000000000	-512 GB	ffffffeeffffffff	444 GB	Khoảng trống chưa sử dụng
ffffffef00000000	-68 GB	fffffffeffffffff	64 GB	Không gian ánh xạ vùng EFI
fffffffff0000000	-4 GB	fffffffff7ffffff	2 GB	Khoảng trống chưa sử dụng
fffffffff8000000	-2 GB	fffffffff9ffffff	512 MB	Ánh xạ mã lệnh nhân (*kernel text*, `phys=0`)
fffffffffa0000000	-1536 MB	ffffffffffefffff	1520 MB	Không gian ánh xạ các module nhân
ffffffffff000000	-16 MB	ffffffffff5fffff	~0.5 MB	Vùng `fixmap` nội bộ của nhân
ffffffffff600000	-10 MB	ffffffffff600fff	4 KB	Giao diện `vsyscall` thế hệ cũ
ffffffffffe00000	-2 MB	ffffffffffffffff	2 MB	Khoảng trống chưa sử dụng

B Phụ lục: Các vùng nhớ ảo macOS (MacOS Virtual Memory Regions)

Bố cục này cung cấp cái nhìn có cấu trúc về cách bộ nhớ được cấp phát và bảo vệ trong không gian địa chỉ tiến trình của macOS:

Vùng (Region)	Địa chỉ / Phạm vi	Kích thước	Quyền hạn	Chia sẻ	Ghi chú
`==== Các vùng không cho phép ghi (Non-writable regions) của tiến trình xxx`					
`__PAGEZERO`	`0`	4K	`--/--`	NUL	Trang bẫy con trỏ NULL
`__TEXT`	`1000`	40K	`r-x/rwx`	COW	Mã thực thi chính (Clock)
`__LINKEDIT`	`e000`	4K	`r-/rwx`	COW	Bảng liên kết động
`Submap`	`90000000-9fffffff`	—	`r-/r-`	—	Bản đồ con toàn máy (thư viện hệ thống dùng chung)
`__TEXT`	`90000000`	932K	`r-x/r-x`	COW	`libSystem.B.dylib`
`__TEXT`	`90130000`	740K	`r-x/r-x`	COW	`CoreFoundation`
`__TEXT`	`90220000`	2144K	`r-x/r-x`	COW	`CarbonCore`
`==== Các vùng cho phép ghi (Writable regions) của tiến trình xxx`					
`__DATA`	`18000`	4K	`rw-/rwx`	PRV	Dữ liệu ứng dụng (TextEdit)
`__OBJC`	`19000`	8K	`rw-/rwx`	COW	Dữ liệu Objective-C
`MALLOC_OTHER`	`1d000`	256K	`rw-/rwx`	PRV	Vùng cấp phát Heap khác
`MALLOC_USED`	`5d000`	256K	`rw-/rwx`	PRV	Vùng Heap đang sử dụng
`VALLOC_USED`	`ff000`	36K	`rw-/rwx`	PRV	Vùng cấp phát theo trang
`Submap`	`a000b000-a012ffff`	—	`r-/r-`	—	Chỉ dành riêng cho tiến trình
`STACK[0]`	`bfffe000`	4K	`rw-/rwx`	PRV	Ngăn xếp luồng 0
`STACK[1]`	`f0001000`	512K	`rw-/rwx`	PRV	Ngăn xếp luồng 1

C Phụ lục: Bố cục Địa chỉ NTKernel (Windows NTKernel Address Layout)

Địa chỉ bắt đầu	Địa chỉ kết thúc	Kích thước	Mô tả vùng nhớ nhân
`FFFF0800‘00000000`	`FFFFF67F‘FFFFFFFF`	238TB	Không gian hệ thống chưa sử dụng
`FFFFF680‘00000000`	`FFFFF6FF‘FFFFFFFF`	512GB	Không gian lưu bảng trang (PTE Space)
`FFFFF700‘00000000`	`FFFFF77F‘FFFFFFFF`	512GB	Không gian siêu ánh xạ (HyperSpace)
`FFFFF780‘00000000`	`FFFFF780‘00000FFF`	4K	Trang hệ thống dùng chung (Shared System Page)

Địa chỉ bắt đầu	Địa chỉ kết thúc	Kích thước	Mô tả vùng nhớ nhân
FFFFF780‘00001000	FFFFF7FF‘FFFFFFFF	512GB-4K	Tập làm việc bộ nhớ đệm hệ thống (System Cache Working Set)
FFFFF800‘00000000	FFFFF87F‘FFFFFFFF	512GB	Ánh xạ bộ nạp khởi tạo (Initial Loader Mappings)
FFFFF880‘00000000	FFFFF89F‘FFFFFFFF	128GB	Các PTE hệ thống (Sys PTEs)
FFFFF8a0‘00000000	FFFFF8bF‘FFFFFFFF	128GB	Vùng bộ nhớ nhân có thể hoán đổi (Paged Pool Area)
FFFFF900‘00000000	FFFFF97F‘FFFFFFFF	512GB	Không gian phiên làm việc (Session Space)
FFFFF980‘00000000	FFFFFFa70‘FFFFFFFF	1TB	Không gian địa chỉ ảo nhân động (Dynamic Kernel VA Space)
FFFFFa80‘00000000	*nt!MmNonPagedPoolStart-1	Tối đa 6TB	Cơ sở dữ liệu số khung trang (PFN Database)
*MmNonPagedPoolStart	*MmNonPagedPoolEnd	Tối đa 512GB	Vùng nhớ nhân cố định trên RAM (Non-Paged Pool)
FFFFFFFF‘FFc00000	FFFFFFFF‘FFFFFFFF	4MB	Ánh xạ lớp trừu tượng phần cứng HAL và Loader

D Phụ lục: Nguyên mẫu phần mềm của bộ máy DMA (Software prototype of the DMA engine)

```
int _dma_copy(int bank, uint64_t src_offset,
              uint64_t dst_offset, uint64_t length) {
    DEBUG_PRINT("DMA CALL: bank=%d, src=%llu, dst=%llu, len=%llu\n",
                bank, src_offset, dst_offset, length);

    /* Gia su dinh dia chi BUS duoc cung cap qua ham get_ptr() */
    void *src = get_ptr((size_t)src_offset);
    void *dst = get_ptr((size_t)dst_offset);

    DEBUG_PRINT("Before copy:\n");
    DEBUG_PRINT("  src[0] = 0x%02x, src[1] = 0x%02x\n",
                ((uint8_t*)src)[0], ((uint8_t*)src)[1]);
    DEBUG_PRINT("  dst[0] = 0x%02x, dst[1] = 0x%02x\n",
                ((uint8_t*)dst)[0], ((uint8_t*)dst)[1]);

    DEBUG_PRINT("\nCalling DMA copy directly...\n");

    if (src && dst) {
        memcpy(dst, src, (size_t)length);
    }

    DEBUG_PRINT("\nAfter copy:\n");
    DEBUG_PRINT("  src[0] = 0x%02x, src[1] = 0x%02x\n",
                ((uint8_t*)src)[0], ((uint8_t*)src)[1]);
    DEBUG_PRINT("  dst[0] = 0x%02x, dst[1] = 0x%02x\n",
                ((uint8_t*)dst)[0], ((uint8_t*)dst)[1]);

    return src && dst ? 0 : -1;
```

```
29 }
```

Listing 1: Nguyên mẫu phần mềm của DMA (DMA software prototype)